Neural Networks

AERO 689: Machine Learning for Aerospace Engineers

Raktim Bhattacharya

Texas A&M University

Table of contents

Why Neural Networks?

Motivation

Within data-driven ML, when should we use neural networks over classical methods?

Classical methods (Weeks 2–6): linear/regularized regression, logistic regression, PCA, POD, clustering

- These methods are **not inferior** — for many problems they are the right choice
- Ridge regression outperforms a NN on a 50-sample aerodynamic dataset
- Neural networks earn their place only when the problem **exceeds what the classical toolkit handles gracefully**

What Classical ML Cannot Do Easily

Limitation	Why it matters in aerospace
Fixed feature map — hand-engineering required	Wing pressure field: $O(10^5)$ DOF — no human can specify the right features
Shallow representation — hierarchical structure is expensive	Aero-acoustic noise: vortex roll-up $\rightarrow$ shear layer $\rightarrow$ far-field radiation
Does not scale — $n \times n$ matrices become intractable	Hyperspectral imagery, LiDAR, full-field strain maps
No structure sharing — inputs treated independently	Defect detection: pixel neighborhoods carry info a flat vector discards
No sequence memory — samples treated as i.i.d.	Flight dynamics, fault evolution over time, ATC commands

The Neural Network's Answer

1. Automatic feature learning — first layers extract low-level features; deeper layers compose abstractions. No domain expert needed to specify them.

2. Depth and compositionality [1], [2] — deep networks are *compact*: exponentially more efficient than shallow models for hierarchical functions.

3. Inductive biases that match physical structure

- **CNNs** — local, translation-invariant patterns (defect detection, pressure grids)
- **RNNs / LSTMs** — sequential data (flight dynamics, engine health trending)
- **GNNs** — unstructured meshes and sensor networks

4. Scaling laws [3] — classical $O(n^2)$–$O(n^3)$ methods are impractical for large n; NNs with SGD scale to billions of samples.

Why NNs Scale: SGD

Classical methods require the full dataset simultaneously:

Method	Cost per solve	Memory
Least squares	$O(np^2 + p^3)$	$O(np)$
Kernel / GP	$O(n^3)$	$O(n^2)$

SGD replaces the full gradient with a mini-batch estimate ($|\mathcal{B}| = m \ll n$):

$$\nabla_\theta \mathcal{L} \approx \frac{1}{m} \sum_{i \in \mathcal{B}} \nabla_\theta \, \ell(f_\theta(x_i),\, y_i)\,, \qquad \text{cost per step: } O(m \cdot p)$$

- Cost per update is **constant** in n — same whether $n = 10^4$ or 10^{10}
- NN loss **decomposes** as a sum over samples $\Rightarrow$ per-sample gradients are independent
- Backprop computes $\nabla_\theta \ell$ in $O(p)$ — no matrix inversion, no kernel matrix
- Mini-batch noise acts as **implicit regularization** (helps escape sharp minima)

Limitations: Optimization and Constraints

Optimization

- Loss is **non-convex** — SGD finds local minima; no global optimality guarantee
- Convergence is not guaranteed; sensitive to learning rate and initialization
- Classical methods (ridge, SVM, GP) solve *convex* problems with guaranteed global optima

Constraints

- No native constraint handling — classical methods satisfy $g(x) = 0$ exactly via KKT / Lagrange multipliers
- NNs must impose constraints via:
  - **Architecture** (hard) — output layer designed to enforce bounds or symmetry; exact but inflexible
  - **Penalty terms** (soft) — add $\lambda\, g(\theta)^2$ to the loss; satisfied only *approximately*; the weight λ requires tuning

Limitations: Data, UQ, and Hyperparameters

Data and compute

- **Data hungry** — classical methods generalize with $n \sim 10^2$; NNs typically need $n \gtrsim 10^4$
- Training large models requires GPUs and wall-clock time that classical solvers do not

Interpretability and uncertainty quantification

- **Black box** — individual weights carry no physical meaning; predictions are hard to audit
- **No built-in uncertainty** — linear regression gives exact confidence intervals; NNs require ensembles, MC dropout, or full Bayesian NNs to produce calibrated uncertainty estimates

Hyperparameters

- Architecture depth/width, learning rate, batch size, regularization strength, and optimizer must all be tuned
- No closed-form selection — typically requires cross-validation or neural architecture search (NAS)

When to Prefer Classical ML

Prefer classical methods when:

- **Small dataset** ($n \lesssim 10^3$): Ridge/Lasso generalizes better and gives exact confidence intervals
- **Smooth, low-dimensional response**: splines or polynomial regression — interpretable and sufficient
- **Interpretability required**: linear models / decision trees produce auditable rules
- **Tabular data with engineered features**: gradient boosted trees (XGBoost) consistently outperform NNs
- **Closed-form statistics needed**: linear regression gives exact confidence intervals; NNs require ensembles or dropout for uncertainty

Rule of Thumb

Use a neural network when **at least one** of the following holds:

1. Input is **raw and high-dimensional** — images, time series, fields, point clouds
2. Function is **highly nonlinear and hierarchical** — cannot be captured by a fixed kernel
3. **Sufficient data** — $n \gtrsim 10^4$, or transfer learning from a pretrained model is available
4. **Temporal or spatial structure** should be exploited automatically (RNN/LSTM, CNN)

Otherwise, start with the simplest model that works.

Neural Networks

A Neural Network as a Function

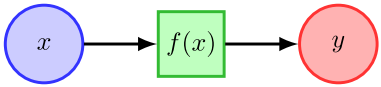


Figure 1: Simple: $y := f(x)$

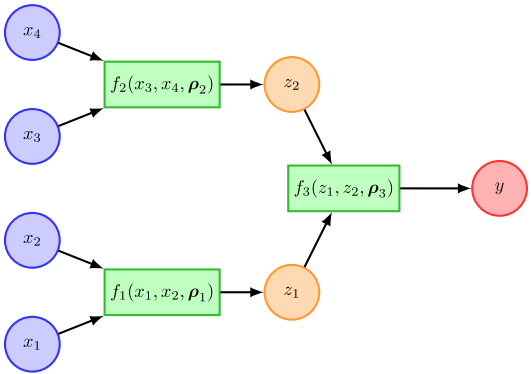
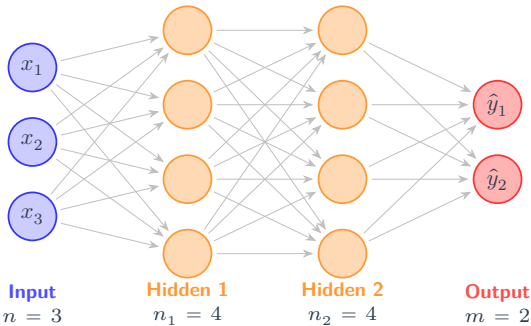


Figure 2: Complex: $y = f_3(f_1(\cdot),\ f_2(\cdot),\ \rho_3)$

In general: $y = f(x, \rho)$ where $x \in \mathcal{R}^n$ is the input and $\rho \in \mathcal{R}^p$ are the parameters.

Dense Feedforward Network

Every neuron is connected to every neuron in the next layer — hence *dense*.



Layer Operations

Each layer ℓ computes the **vector of activations**:

$$\underbrace{a^{(\ell)}}_{\text{output of layer } \ell} = \sigma\Big(W^{(\ell)}\, a^{(\ell-1)} + b^{(\ell)}\Big)$$

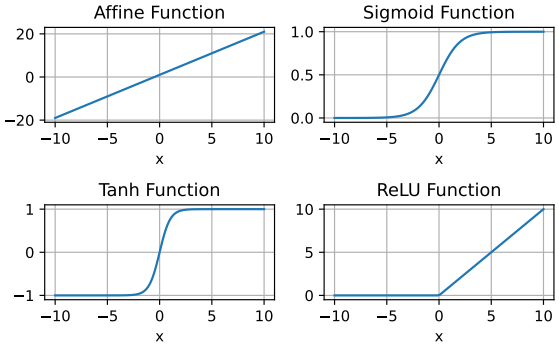
- $W^{(\ell)}$ — weight matrix; $b^{(\ell)}$ — bias vector
- σ — **activation function**, applied element-wise (broadcasting)
- "Activation of a vector" = "vector of activations" — same computation, different perspective
- The network is a *graphical architecture for composing parameterized nonlinear functions*

Activation Functions

Common Activation Functions

Name	Function
Affine	$W^T x + b$
Sigmoid	$\frac{1}{1+e^{-x}}$
Tanh	$\tanh(x)$
ReLU	$\max(0, x)$

Non-linear activations are essential: a composition of linear functions is still linear.



Building Neural Networks in Python

Framework Overview

Library	Backend	Primary use	By
TensorFlow / Keras	TF	Production, deployment	Google
PyTorch	PyTorch	Research, custom arch	Meta
scikit-learn MLP	NumPy	Rapid prototyping	Community

In practice: TensorFlow/Keras and PyTorch dominate.

TensorFlow / Keras

```
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense

fhat = Sequential([
    Dense(2,  input_dim=1),
    Dense(10, activation="tanh"),
    Dense(5,  activation="tanh"),
    Dense(1,  activation="sigmoid"),
])

# Reverse-mode AD
x = tf.linspace(-10, 10, 100)
with tf.GradientTape() as tape:
    tape.watch(x)
    y = fhat(x)
dydx = tape.gradient(y, x)
```

PyTorch: Model Definition

```
import torch
import torch.nn as nn

class FHat(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, 2),
            nn.Linear(2, 10), nn.Tanh(),
            nn.Linear(10, 5), nn.Tanh(),
            nn.Linear(5, 1),  nn.Sigmoid(),
        )
    def forward(self, x):
        return self.net(x)

fhat = FHat()
```

PyTorch: Automatic Differentiation

```
x    = torch.linspace(-10, 10, 100,
                      requires_grad=True).unsqueeze(1)
y    = fhat(x)
y.sum().backward()       # dy/dx for all x simultaneously
dydx = x.grad
```

	TensorFlow / Keras	PyTorch
Training loop	`model.fit()` (hidden)	Explicit `loss.backward()`
Flexibility	High-level, concise	Fully customizable
Best for	Production / deployment	Research / custom arch

Universal Approximation Theory

Universal Approximation Theorem

Theorem (Cybenko [4]; Hornik [5]). A feedforward network with one hidden layer and finitely many neurons can approximate *any* continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy.

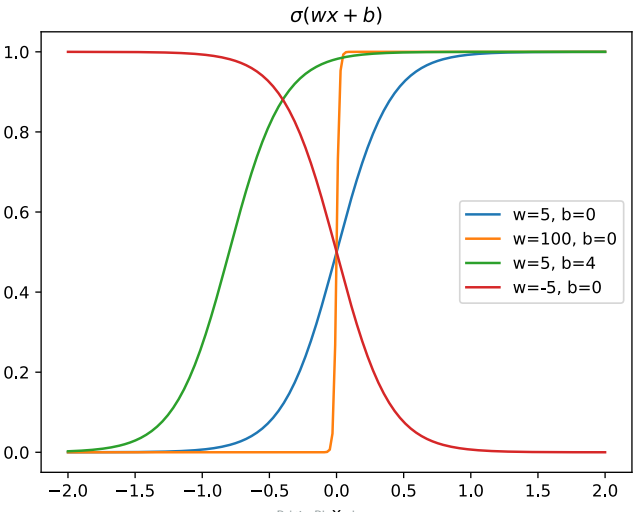
Key implications:

1. **Expressiveness** — NNs are not limited in the functions they can represent
2. **Depth efficiency** — deeper networks achieve the same accuracy with exponentially fewer parameters
3. **Non-linear activation is essential** — a composition of linear functions is still linear
4. **Does not address**: how to train, convergence speed, overfitting, or architecture choice

The theorem guarantees existence — not learnability.

Impact of Weights and Biases

Effect of Weights and Biases



Neural Basis Functions

By linearly combining sigmoid and affine functions — **locally supported functions**:

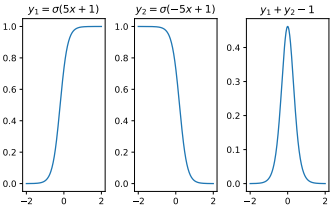


Figure 4: Bell: $y_1 + y_2 - 1$

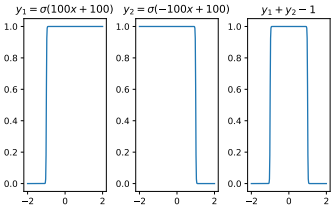


Figure 5: Box: large w

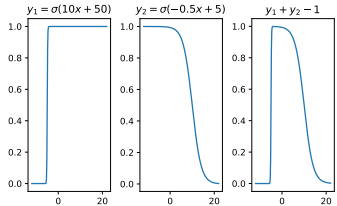


Figure 6: Asymmetric: $w_1 \neq w_2$

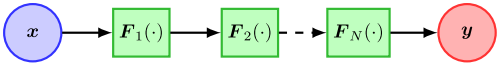
Optimizing weights and biases constructs a rich library of neural basis functions.

Deep Neural Networks

Feed-Forward Neural Network (FFNN / MLP)

$$F(x) = (F_N \circ \cdots \circ F_1)(x)$$

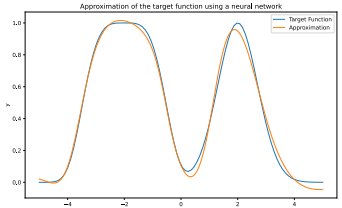
$$F_i(\cdot) = f_i(W_i\, F_{i-1}(\cdot) + b_i)$$



- W_i, b_i: weight matrix and bias of layer i
- f_i: activation function of layer i
- Well-suited for tabular data and supervised learning
- Address overfitting via weight decay or dropout
- See [6, Ch. 6] for a thorough treatment

FFNN Example: TensorFlow

```
def target_function(x):
    return np.exp(-(x-2)**2) + np.exp(-0.15*(x+2)**4)

x_train = np.linspace(-5, 5, 100).reshape(-1, 1)
y_train = target_function(x_train)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='tanh', input_shape=(1,)),
    tf.keras.layers.Dense(32, activation='tanh'),
    tf.keras.layers.Dense(32, activation='tanh'),
    tf.keras.layers.Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(x_train, y_train, epochs=1000)
```



FFNN Example: PyTorch

```
class FFNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, 32), nn.Tanh(),
            nn.Linear(32, 32), nn.Tanh(),
            nn.Linear(32, 32), nn.Tanh(),
            nn.Linear(32, 1),
        )
    def forward(self, x): return self.net(x)

model     = FFNN()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn   = nn.MSELoss()

for epoch in range(5000):
    optimizer.zero_grad()
    loss = loss_fn(model(x_train), y_train)
    loss.backward()
    optimizer.step()
```

Application: Aerodynamic Surrogate Modeling

Problem: Predict $\hat{C}_D(M, \alpha)$ — CFD requires hours per operating point.

Approach	Fails at	Advantage
Linear regression	Transonic drag rise ($M \approx 0.8$), stall ($\alpha > 12^\circ$)	Interpretable; fast
FFNN (2→64→64→1)	Extrapolation past training envelope	Captures strong nonlinearities

Inputs: $[M, \alpha]$ **Output:** C_D **Data:** CFD sweep or wind-tunnel runs

Aerodynamic Surrogate: PyTorch

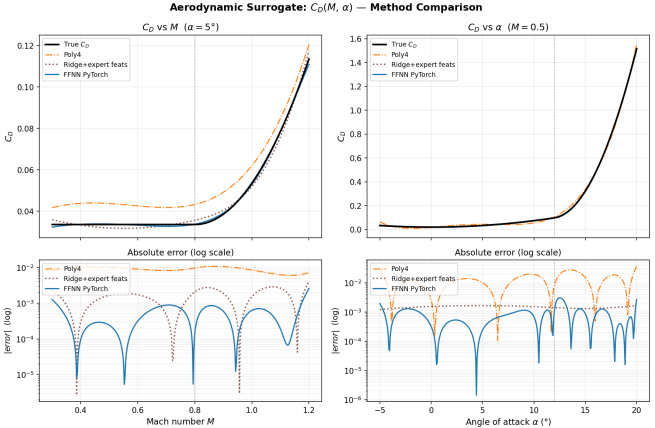
```
import torch, torch.nn as nn

class DragSurrogate(nn.Module):       # [M, alpha] -> C_D
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 64), nn.Tanh(),
            nn.Linear(64, 64), nn.Tanh(),
            nn.Linear(64, 1),
        )
    def forward(self, x): return self.net(x)

model     = DragSurrogate()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn   = nn.MSELoss()

# X_train: (N, 2) [M, alpha];  y_train: (N, 1) C_D
for epoch in range(5000):
    optimizer.zero_grad()
    loss = loss_fn(model(X_train), y_train)
    loss.backward()
    optimizer.step()
```

Aerodynamic Surrogate: Accuracy Results

Method	RMSE	R^2
Polynomial (deg 4)	0.0142	0.998
Ridge + expert feats	0.0027	1.000
FFNN PyTorch	**0.0024**	**1.000**

- 2,400 train / 600 test.
- $C_D \in [0.015,\ 1.52]$.

Takeaway: expert features beat the FFNN — NNs earn their place when you *don't* know which features to engineer.



Top: predictions; bottom: |error| (log scale). Poly4 and expert features track well; NNs match without feature engineering.

Specific Deep Learning Architectures

Sequence Prediction: Embedding the Past

Many aerospace problems are **sequential**: given a history of observations $x_1, x_2, \ldots, x_t$, predict what happens next.

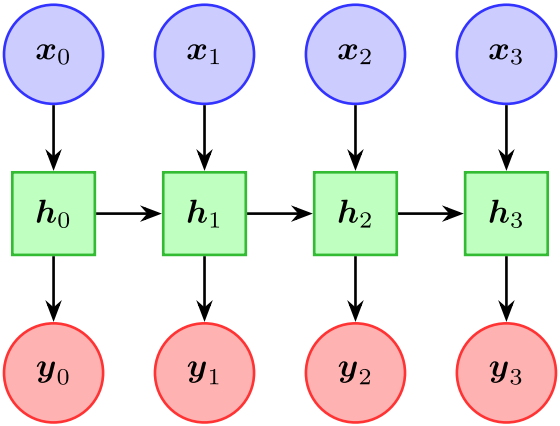
Application	Sequence (past/present)	Prediction (future)
Engine health	Sensor readings over T flights	Remaining useful life
Flight dynamics	States (α, q, V) over time	Next-step trajectory
Weather / turbulence	Gridded fields at $t_{-n} \ldots t_0$	Field at t_1

Core challenge: an FFNN takes a fixed-size input. How do we compress a variable-length sequence into a fixed representation — a **sequence embedding** — that retains the information needed for prediction?

Three approaches, each building on the last:

1. **RNN** — fold the sequence into a hidden state, one step at a time
2. **LSTM** — add gated memory so the embedding can retain long-range information
3. **Transformer** — let every step attend to every other step in parallel

Recurrent Neural Networks (RNN)



$$h_t = F(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = g(W_{hy}h_t + b_y)$$

- Parameters shared across time steps
- **Hidden state** h_t is the sequence embedding — it accumulates information from $x_1, \ldots, x_t$
- At any time t, h_t is a fixed-size summary of everything seen so far

The Vanishing Gradient Problem

Training an RNN requires back-propagation through time (BPTT): gradients flow backward through every time step.

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

The product $\prod \frac{\partial h_k}{\partial h_{k-1}}$ involves multiplying the same weight matrix $T - t$ times:

- If eigenvalues < 1: gradients **vanish** $\Rightarrow$ early time steps are forgotten
- If eigenvalues > 1: gradients **explode** $\Rightarrow$ training diverges

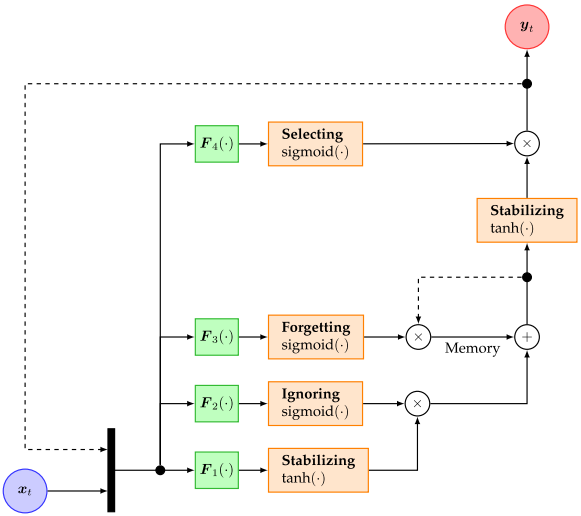
Vanishing Gradients: Consequence and Solution

Consequence: vanilla RNNs struggle to learn dependencies longer than $ $10–20 steps. The sequence embedding h_t loses early information.

Example: predicting engine failure from 200 cycles of sensor data — by the time the RNN reaches cycle 200, the gradient signal from cycle 1 has effectively disappeared.

Solution: add explicit memory with **gates** that control what to remember and what to forget $\Rightarrow$ **LSTM**.

Long Short-Term Memory (LSTM)



Three **gates** regulate memory [7]:

- **Forget gate** f_t — erases irrelevant memory
- **Input gate** i_t — writes new information
- **Output gate** o_t — controls what passes to h_t

The **cell state** c_t acts as a conveyor belt: gradients can flow through it unchanged, solving the vanishing gradient problem.

The sequence embedding is now (h_t, c_t) — richer than the RNN's h_t alone.

Periodic Forecasting Example

Goal: predict the next value of a periodic signal from a window of recent history.

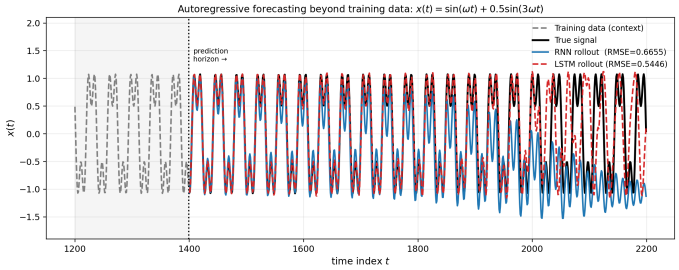
$$x_t = \sin(\omega t) + 0.5\sin(3\omega t), \quad \omega = 0.17$$

$$\hat{x}_{t+1} = f(x_{t-W+1}, \ldots, x_t), \quad W = 30$$

- Train: $t = 0, \ldots, 1399$; test rollout: $t = 1400, \ldots, 2199$
- Compare **vanilla RNN** and **LSTM** (same hidden size $= 32$)
- **Autoregressive rollout**: each prediction is fed back as the next input — no true future values used

```
def signal(t):
    return np.sin(omega * t) + 0.5 * np.sin(3 * omega * t)

X, y = make_windows(signal(t_train), window=30)

rnn  = nn.RNN(input_size=1,  hidden_size=32, batch_first=True)
lstm = nn.LSTM(input_size=1, hidden_size=32, batch_first=True)
```

Periodic Forecasting Results: RNN vs LSTM



- Gray region: last 200 training steps (context). Dashed line: end of training data.
- Both models roll out **800 steps** feeding only their own predictions — no true future values.
- Look-ahead is valid only for inputs known in advance (e.g., planned controls); using unknown future targets is leakage.
  - Valid only when future signals are genuinely available at inference time.

RNN: diverges quickly. The fixed-size hidden state h_t cannot retain phase information over many steps.

LSTM: stays accurate near the horizon, but also drifts eventually. The cell state slows error growth but cannot prevent it.

Key insight: autoregressive rollout accumulates errors at every step. Even the best recurrent model has a finite prediction horizon — this is a fundamental limitation, not just a training issue.

From Recurrence to Attention

LSTM processes the sequence **one step at a time** — step 50 must wait for steps 1–49. Two limitations:

1. **No parallelism** — training is slow on long sequences
2. **Information bottleneck** — all history must squeeze through the fixed-size state (h_t, c_t)

Key insight [8]: instead of passing information forward through a chain of hidden states, let every position in the sequence **look at every other position directly**.

Self-Attention Mechanism

Given an input sequence $x_1, \ldots, x_T$, compute three projections per position:

- $\mathbf{Q}$ (Query) — "what am I looking for?"
- $\mathbf{K}$ (Key) — "what do I contain?"
- $\mathbf{V}$ (Value) — "what information do I pass along?"

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

The softmax produces a **weight matrix** over all positions — each output is a weighted combination of all inputs, where the weights are learned from data.

Dividing by $\sqrt{d_k}$ prevents dot products from growing too large in high dimensions.

Transformers: Architecture

A Transformer block stacks: **self-attention** $\rightarrow$ **add & norm** $\rightarrow$ **feedforward** $\rightarrow$ **add & norm**.

- **Multi-head attention** — run h attention operations in parallel on different subspaces, then concatenate. Each head can learn a different relationship (e.g., one head tracks degradation trend, another tracks operating regime).
- **Positional encoding** — since there is no recurrence, position information is injected via sinusoidal or learned embeddings added to the input.
- The sequence embedding for position t is an **attention-weighted combination** of all positions — richer than LSTM's fixed-size state.

Sequence Models: Comparison

	RNN	LSTM	Transformer
Long-range memory	Poor	Good	Excellent
Parallel training	No	No	Yes
Sequence embedding	h_t	(h_t, c_t)	attention-weighted
Complexity per layer	$O(T \cdot d^2)$	$O(T \cdot d^2)$	$O(T^2 \cdot d)$

- **RNN:** simple, fast per step, but forgets early inputs
- **LSTM:** gated memory solves vanishing gradients; standard for moderate-length sequences
- **Transformer:** best for long-range dependencies and large-scale data; $O(T^2)$ cost per layer

Active research: linear attention, sparse attention, and state-space models aim to reduce the $O(T^2)$ cost while keeping the benefits of attention.

RNN Example: Frequency Estimation

```
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(50, input_shape=(50, 1), return_sequences=True),
    SimpleRNN(50),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=10, batch_size=32, validation_split=0.2)
```

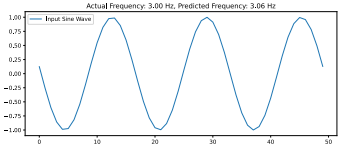


Figure 7: Frequency classification from sine-wave time series.

Application: Engine Health — NASA C-MAPSS

C-MAPSS = Commercial Modular Aero-Propulsion System Simulation [9].

NASA simulated turbofan engines running from healthy to failure under realistic conditions.

- One **cycle** = one start → operate → shutdown sequence (i.e. one flight).
- Each row in the dataset is a snapshot of all sensors during one cycle.
- Each engine has a **different initial wear** and **unknown fault onset**, so trajectories vary in length (e.g. 128–362 cycles).

Item	Description
Subset used	FD001 — single operating condition, single fault mode (HPC degradation)
Training / Test	100 / 100 engines (run-to-failure in train; cut-off in test)
Sensor channels	21 (temperatures, pressures, fan/core speeds, bypass ratio, …)
Operating settings	3 (altitude, Mach, throttle resolver angle)
Target	Remaining Useful Life (RUL) — cycles until failure, capped at 125

Free download: `https://ti.arc.nasa.gov/c/6/`

Engine Health: Problem Setup

Goal: predict how many cycles an engine has left before failure.

- **Piece-wise linear RUL:** true RUL decreases linearly (299, 298, ...), but early in life the sensors look identical — no degradation signal.
  - Capping at 125 means: any RUL > 125 is set to 125, so the model only learns the **degradation phase** where sensors actually change.
- **Rolling window ($w = 30$ cycles):** at each time step we feed the last 30 sensor readings to the model.

Inputs	Output	Why LSTM?
14 sensors + 3 settings over 30 cycles (30×17 matrix)	RUL (cycles)	Hidden state captures gradual degradation trend across the window

Engine Health: Classical Baseline

Strategy: Extract hand-crafted features from a 30-cycle window, then Ridge regression with polynomial interaction terms.

```
# For each window of 30 cycles, extract:
#   rolling mean, rolling std, trend (Δ), cycle number
# per sensor channel → 52 features per sample

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import RidgeCV

ridge = Pipeline([
    ("poly", PolynomialFeatures(degree=2, interaction_only=True)),
    ("scl",  StandardScaler()),
    ("ridge", RidgeCV(alphas=[0.01, 0.1, 1.0, 10.0, 100.0])),
]).fit(X_train, y_train)
```

52 hand-crafted features $\rightarrow$ 1,378 polynomial interaction terms $\rightarrow$ Ridge selects the best linear combination.

Engine Health: Feature Engineering

A 30-cycle window of raw sensor data is a 30×17 matrix. Ridge regression needs a flat vector, so we summarize each sensor column:

Feature	What it captures
Rolling mean	Current average state of the sensor
Rolling std	How noisy / volatile the sensor is
Trend (Δ)	Is the sensor drifting up or down? (degradation direction)
Cycle number	Where in the engine's life we are

This yields $\sim$ 52 scalar features per sample — but **discards the time ordering** within the window.

Engine Health: Why Polynomial Ridge?

Polynomial interactions (`degree=2, interaction_only=True`) — creates all pairwise products of the 52 features, e.g. $\overline{T}_{30} \times \sigma_{\mathrm{P}_{15}}$.

- This lets a linear model capture nonlinear patterns like "temperature rising *while* pressure becomes noisy $\Rightarrow$ imminent failure."
- 52 features $\rightarrow$ 1,378 interaction terms

Ridge (L2) regularization prevents overfitting on 1,378 terms; `RidgeCV` auto-selects the penalty strength via cross-validation.

Key limitation: the temporal **sequence** inside each window is lost — LSTM addresses this.

Engine Health: LSTM (PyTorch)

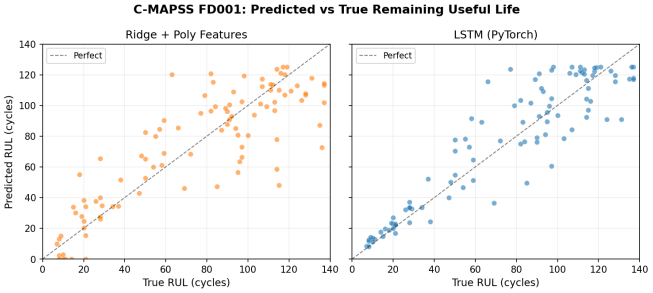
Strategy: Feed raw 30-cycle sensor sequences directly — let the LSTM learn temporal features.

```
class RULPredictor(nn.Module):
    def __init__(self, n_feat=17, h1=64, h2=32):
        super().__init__()
        self.lstm1 = nn.LSTM(n_feat, h1, batch_first=True)
        self.lstm2 = nn.LSTM(h1, h2, batch_first=True)
        self.fc    = nn.Linear(h2, 1)

    def forward(self, x):                # x: (batch, 30, 17)
        out, _ = self.lstm1(x)
        out, _ = self.lstm2(out)
        return self.fc(out[:, -1, :])    # last hidden → RUL

model     = RULPredictor()
optimizer = torch.optim.Adam(model.parameters(), lr=5e-4)
for epoch in range(80):
    for X_b, y_b in train_loader:
        optimizer.zero_grad()
        loss = nn.MSELoss()(model(X_b), y_b)
        loss.backward()
        optimizer.step()
```

Engine Health: Results

Method	RMSE	R^2
Ridge + poly feats	21.5	0.73
LSTM	**17.2**	**0.83**

- LSTM: **20% lower RMSE** — no feature engineering needed
- Ridge needs domain knowledge to build 52 hand-crafted features
- LSTM learns temporal patterns directly from raw sensors



Predicted vs true RUL (cycles). LSTM points cluster tighter around the diagonal.

Other Architectures

Autoencoders — encoder compresses input to a latent space; decoder reconstructs. Used for dimensionality reduction and anomaly detection.

Convolutional Neural Networks (CNNs) [10] — convolutional layers extract local spatial features with weight sharing. Backbone of image recognition; applied to damage detection and pressure-field processing.

Generative Adversarial Networks (GANs) [11] — generator vs. discriminator trained adversarially. Used for data augmentation and generative design.

Application: Damage Detection and Generative Design

Architecture	Aerospace use case	Key advantage
CNN	Ultrasonic C-scan: delamination detection in composites	Local spatial features; weight sharing
GAN	Synthetic airfoil shape generation for data augmentation	Diverse samples without new CFD runs
Transformer	Neural operator: BCs $\rightarrow$ full PDE field solution	Long-range spatial dependencies

Below: CNN delamination detector on 64$\times$64 ultrasonic C-scan images (2 classes: intact / delaminated).

Damage Detection: PyTorch

```
import torch, torch.nn as nn

class DamageDetector(nn.Module):           # 64x64 C-scan -> class label
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 16 * 16, 128), nn.ReLU(),
            nn.Linear(128, 2),             # 2 classes: intact / delaminated
        )
    def forward(self, x): return self.classifier(self.features(x))

model     = DamageDetector()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn   = nn.CrossEntropyLoss()

for epoch in range(30):
    for X_b, y_b in dataloader:            # X_b: (B, 1, 64, 64)
        optimizer.zero_grad()
        loss = loss_fn(model(X_b), y_b)
        loss.backward()
        optimizer.step()
```

Summary

Architecture Overview

Architecture	Best suited for	Key characteristic
FFNN / MLP	Tabular data, regression	Universal approximator; fully connected
RNN	Sequences, time series	Hidden-state recurrence; temporal memory
LSTM	Long sequences, forecasting	Gated memory; avoids vanishing gradients
Autoencoder	Dim. reduction, anomaly detect	Encoder–decoder; unsupervised
CNN	Images, spatial fields	Local receptive fields; weight sharing
GAN	Generative modeling	Adversarial training
Transformer	Long sequences, operator learning	Self-attention; parallel; no recurrence

NNs are applied across aerospace: surrogate modeling, structural health monitoring, turbulence closure, trajectory optimization, and guidance & control.

References

[1] M. Telgarsky, "Benefits of depth in neural networks," in *Proceedings of the 29th annual conference on learning theory (COLT)*, 2016, pp. 1517–1539.

[2] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao, "Why and when can deep—but not shallow—networks avoid the curse of dimensionality: A review," *International Journal of Automation and Computing*, vol. 14, no. 5, pp. 503–519, 2017.

[3] J. Kaplan *et al.*, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[4] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.

References

[5] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] A. Vaswani *et al.*, "Attention is all you need," in *Advances in neural information processing systems (NeurIPS)*, 2017.

[9] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *International conference on prognostics and health management (PHM)*, IEEE, 2008, pp. 1–9.

References

[10] Y. LeCun *et al.*, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.

[11] I. Goodfellow *et al.*, "Generative adversarial nets," in *Advances in neural information processing systems (NeurIPS)*, 2014.